LIVRET DE L'APPRENANT

Suivi d'incidents sur un réseau backbone

Apprendre Laravel, MySQL et le développement web
en construisant un projet concret, en 8 semaines.

Semaine 1	Cadrage métier et environnement
Semaine 2	La base de données MySQL
Semaine 3	Blade, Bootstrap et authentification
Semaine 4	Inventaire réseau (CRUD)
Semaine 5	Le cœur de l'application — les incidents
Semaine 6	Journal de suivi, recherche et export
Semaine 7	Tableau de bord, rapport hebdomadaire et rôles
Semaine 8	Tests, finitions et soutenance

Laravel · Blade · Bootstrap 5 · MySQL

LIVRET DE L'APPRENANT

Bienvenue dans le projet

Pendant 8 semaines, tu vas construire une **vraie application** utilisée dans un NOC (centre de supervision réseau) : le suivi des incidents d'un réseau backbone. Chaque semaine, tu reçois une fiche qui t'apprend une partie de **Laravel**, de **MySQL** et du **métier de développeur**, puis te fait l'appliquer sur le projet.

Le programme

Semaine	Thème	Ce que tu apprends
1	Cadrage métier et environnement	Le métier NOC, les outils, Git, la modélisation
2	La base de données MySQL	SQL, migrations, Eloquent, relations, enums
3	Blade, Bootstrap et authentification	Routes, vues, sessions, sécurité web
4	Inventaire réseau (CRUD)	CRUD, validation, formulaires, requêtes N+1
5	Les incidents	Form Requests, règles métier, dates
6	Journal, recherche et export	Scopes, pagination, index MySQL, CSV
7	Tableau de bord, rapport et rôles	Agrégations, MTTR, semaines ISO, autorisations
8	Tests, finitions et soutenance	Tests automatisés, qualité, présentation

Comment se passe une semaine

1. **Lundi** : tu lis la fiche en entier avant de coder. Tu notes ce que tu ne comprends pas.
2. **Du lundi au jeudi** : tu suis les parties de cours, tu fais les exercices MySQL, puis le travail demandé.
3. **Vendredi** : tu remplis l'auto-évaluation et tu présentes ton travail à ton tuteur (30 min).

Les règles du jeu

- **Stack imposée** : Laravel, Blade, Bootstrap 5, MySQL. Pas de starter kit (Breeze, Jetstream…) : tu écris tout toi-même pour comprendre.
- **Git** : au moins un commit par tâche terminée, avec un message clair (« Ajoute la migration des liaisons »).
- **Lis la documentation** avant de chercher ailleurs : laravel.com/docs et dev.mysql.com/doc sont tes premières ressources.
- **Bloqué plus de 30 minutes ?** Écris précisément ce que tu as essayé, le message d'erreur exact, puis demande de l'aide. Savoir formuler un problème est une compétence de développeur.
- **Les outils d'IA** peuvent t'expliquer une notion, mais tu dois être capable d'expliquer chaque ligne que tu commites.

Comment lire les fiches

- Les blocs encadrés signalent un point **à retenir** ou un **piège** courant.
- Les **exercices MySQL** se font dans le terminal, avec le client `mysql`, sur ta base de développement. Garde tes requêtes dans un fichier `.sql` commité.
- L'**auto-évaluation** n'est pas une note : coche honnêtement, c'est ce qui permet à ton tuteur de t'aider au bon endroit.

Bonne construction !

FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 1

Cadrage métier et environnement

Durée : 5 jours Métier NOC · Outils du développeur · Git · Modélisation

Objectifs de la semaine

À la fin de la semaine, tu sais :

- expliquer ce qu'est un incident sur un réseau backbone et comment un NOC le traite ;
- installer et faire tourner un projet Laravel connecté à MySQL ;
- utiliser Git pour sauvegarder ton travail ;
- dessiner un modèle de données avant d'écrire la moindre ligne de code.

1. Le métier : comprendre avant de coder

Un **backbone** est le réseau cœur d'un opérateur. Il relie les grandes villes par des **liaisons** à fort débit (fibre optique, faisceaux hertziens, satellite). Ces liaisons arrivent dans des **sites** (POP, data centers, stations FH) qui hébergent des **équipements** : routeurs, switchs, multiplexeurs DWDM, firewalls.

Le **NOC** (*Network Operations Center*) surveille ce réseau 24h/24. Quand une alarme tombe, un technicien :

1. **déclare** un incident (que se passe-t-il, où, depuis quand ?) ;
2. le **qualifie** (gravité, catégorie) et l'**assigne** à quelqu'un ;
3. **trace** chaque action (appel au terrain, redémarrage, remplacement de carte…) ;
4. le **résout** puis documente la **cause racine** et la **solution**.

Terme	Signification
P1 – Critique	Coupure de service importante (ex. une ville entière isolée)
P2 – Majeure	Service dégradé, ou redondance perdue
P3 – Mineure	Pas d'impact client direct
Cycle de vie	Ouvert → En cours → Résolu → Clos
MTTR	*Mean Time To Restore* : durée moyenne entre le début d'un incident et son rétablissement

> **À retenir** — Un développeur qui ne comprend pas le métier écrit une application que personne n'utilise. Pose des questions, toujours.

2. Les outils du développeur

Outil	Rôle	Vérifier l'installation
PHP 8.3+	Le langage de Laravel	`php -v`
Composer	Gestionnaire de dépendances PHP	`composer -V`
MySQL 8	Base de données relationnelle	`mysql --version`
Git	Historique et sauvegarde du code	`git --version`
VS Code	Éditeur (extensions : PHP Intelephense, Laravel Blade)	—

Comment fonctionne une requête dans Laravel

```
Navigateur  →  routes/web.php  →  Contrôleur  →  Modèle (MySQL)
                                        ↓
Navigateur  ←  HTML  ←  Vue Blade  ←────┘
```

Quand tu tapes `http://localhost:8000/incidents`, Laravel cherche la **route** `/incidents`, appelle la méthode du **contrôleur** associée, qui récupère les données via un **modèle** et les passe à une **vue** Blade qui produit le HTML.

3. MySQL : premiers pas en ligne de commande

Se connecter :

```
mysql -u root -p
```

Bonne pratique : **ne pas** utiliser `root` dans une application. On crée une base et un utilisateur dédiés :

```
CREATE DATABASE suivi_incident CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'suivi'@'localhost' IDENTIFIED BY 'un_mot_de_passe_solide';
GRANT ALL PRIVILEGES ON suivi_incident.* TO 'suivi'@'localhost';
FLUSH PRIVILEGES;
```

Quelques commandes utiles :

```
SHOW DATABASES;
USE suivi_incident;
SHOW TABLES;
DESCRIBE users;
```

> **Pourquoi `utf8mb4` ?** C'est le vrai UTF-8 complet de MySQL : accents, caractères spéciaux et emojis sont stockés sans erreur. `utf8` tout court est une version incomplète à éviter.

4. Git : sauvegarder son travail

```
git init
git add .
git commit -m "Initialise le projet Laravel"
git log --oneline
```

Règle d'or : **un commit = une petite étape qui fonctionne**, avec un message qui dit ce que fait le commit (« Ajoute la page de connexion », pas « modifs »).

Le fichier `.env` contient des mots de passe : il est déjà dans `.gitignore` et ne doit **jamais** être commité.

5. Modéliser avant de coder

Un **MCD** (modèle conceptuel de données) décrit les entités, leurs attributs et leurs relations. Exemple : *un site possède plusieurs équipements, un équipement appartient à un seul site* → relation **1-N**, la clé étrangère `site_id` est dans `equipements`.

Travail demandé

1. Installer PHP, Composer, MySQL, Git et VS Code.
2. Créer le projet : `composer create-project laravel/laravel suivi-incident`.
3. Créer la base et l'utilisateur MySQL dédiés (section 3).
4. Configurer `.env` : `DB_CONNECTION=mysql`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, et `APP_LOCALE=fr`.
5. Lancer `php artisan migrate` puis `php artisan serve` ; ouvrir `http://localhost:8000`.
6. Dans MySQL, exécuter `SHOW TABLES;` et `DESCRIBE users;` : que vois-tu ? Qui a créé ces tables ?
7. Initialiser Git et faire le premier commit.
8. **Dessiner le modèle de données** (papier ou draw.io) avec au minimum :
   - `users` (nom, e-mail, rôle) ;
   - `sites` (code, nom, ville, type) ;
   - `equipements` (hostname, type, constructeur, IP de management, statut) — appartient à un site ;
   - `liaisons` (code, type, capacité, statut) — relie un **site A** et un **site B** ;
   - `incidents` (titre, description, sévérité, statut, catégorie, début, rétablissement, cause racine, solution) — rattaché à un site, éventuellement à un équipement **ou** une liaison, déclaré par un utilisateur, assigné à un utilisateur ;
   - `suivis` (journal des actions sur un incident).
9. Lister en 10 lignes maximum les écrans de l'application.

Livrable

- Le projet démarre sur `http://localhost:8000` et est connecté à MySQL.
- Le schéma du modèle de données (photo ou PNG dans `docs/`).
- La liste des écrans.

Auto-évaluation

- [ ] `php artisan migrate` s'exécute sans erreur sur MySQL
- [ ] J'utilise un utilisateur MySQL dédié, pas `root`
- [ ] Mon dépôt Git contient au moins un commit, sans le fichier `.env`
- [ ] Mon schéma contient les 6 tables et leurs cardinalités
- [ ] La liaison « entre deux sites » a bien deux clés étrangères vers `sites`
- [ ] Je sais expliquer P1/P2/P3, le cycle de vie et le MTTR à l'oral

Ressources

- Documentation Laravel — Installation : laravel.com/docs/installation
- Documentation Laravel — Structure des dossiers : laravel.com/docs/structure
- MySQL — CREATE USER et GRANT : dev.mysql.com/doc/refman/8.4/en/create-user.html
- Git en 15 minutes : learngitbranching.js.org

Pour aller plus loin

Chercher la différence entre un **incident** et un **problème** au sens ITIL, et noter en 3 lignes ce que ça changerait dans l'application.

FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 2

La base de données MySQL

Durée : 5 jours SQL Migrations Eloquent Relations Enums PHP

Objectifs de la semaine

- écrire des requêtes SQL simples (SELECT, WHERE, ORDER BY, JOIN) ;
- traduire un modèle de données en **migrations** Laravel ;
- comprendre les **clés étrangères** et ce qui se passe à la suppression ;
- créer des **modèles Eloquent** et leurs relations ;
- remplir la base avec des données réalistes (**factories** et **seeders**).

1. Les migrations : le schéma de la base en code

Une migration décrit une modification de la base. Elle est versionnée avec Git : toute l'équipe a la même structure.

```
php artisan make:migration create_sites_table
```

```
Schema::create('sites', function (Blueprint $table) {
    $table->id();                              // BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
    $table->string('code', 20)->unique();      // VARCHAR(20) + index UNIQUE
    $table->string('nom');
    $table->string('ville');
    $table->string('type');
    $table->string('adresse')->nullable();     // peut être NULL
    $table->timestamps();                      // created_at, updated_at
});
```

```
php artisan migrate            # applique les migrations en attente
php artisan migrate:rollback   # annule le dernier lot
php artisan migrate:fresh      # supprime tout et recrée (DEV uniquement !)
```

Exercice — Après `php artisan migrate`, tape `SHOW CREATE TABLE sites;` dans MySQL. Retrouve chaque ligne de la migration dans le SQL généré.

2. Les clés étrangères

```
$table->foreignId('site_id')->constrained()->restrictOnDelete();
```

Cette ligne crée une colonne `site_id` **et** une contrainte : MySQL refusera un `site_id` qui n'existe pas dans `sites`. Le comportement à la suppression du parent est un **choix métier** :

Méthode	Effet quand on supprime le parent	Exemple dans le projet
`restrictOnDelete()`	Suppression **refusée**	Un site qui a des équipements
`nullOnDelete()`	La colonne passe à `NULL`	Un équipement supprimé → l'incident garde son historique
`cascadeOnDelete()`	Les enfants sont **supprimés aussi**	Un incident supprimé → ses suivis disparaissent

Pour une liaison entre deux sites, on nomme les colonnes nous-mêmes :

```
$table->foreignId('site_a_id')->constrained('sites');
$table->foreignId('site_b_id')->constrained('sites');
```

3. Les index

Un index accélère les recherches, comme l'index d'un livre. On indexe les colonnes **souvent filtrées ou triées** : `statut`, `severite`, `debut_at`.

```
$table->string('statut')->default('ouvert')->index();
```

4. Les modèles Eloquent et les relations

```
#[Fillable(['code', 'nom', 'ville', 'type', 'adresse'])]
class Site extends Model
{
    public function equipements(): HasMany
    {
        return $this->hasMany(Equipement::class);
    }
}

class Equipement extends Model
{
    public function site(): BelongsTo
    {
        return $this->belongsTo(Site::class);
    }
}
```

- `#[Fillable]` liste les colonnes qu'on a le droit de remplir en masse (`Site::create($donnees)`). C'est une **protection** : un utilisateur malveillant ne peut pas injecter une colonne non prévue.
- `$site->equipements` renvoie la collection des équipements ; `$equipement->site` renvoie le site.

Quand la clé étrangère n'a pas le nom standard, on la précise :

```
public function siteA(): BelongsTo
{
    return $this->belongsTo(Site::class, 'site_a_id');
}
```

5. Les enums PHP

Plutôt que des chaînes « magiques » éparpillées dans le code, on définit une liste fermée de valeurs :

```
enum Severite: string
{
    case Critique = 'critique';
    case Majeure = 'majeure';
    case Mineure = 'mineure';

    public function label(): string
    {
        return match ($this) {
            self::Critique => 'P1 – Critique',
            self::Majeure => 'P2 – Majeure',
            self::Mineure => 'P3 – Mineure',
        };
    }
}
```

Dans le modèle, un **cast** transforme automatiquement la valeur de la base en enum :

```
protected function casts(): array
{
    return ['severite' => Severite::class, 'debut_at' => 'datetime'];
}
```

`$incident->severite->label()` affiche alors « P1 - Critique ».

6. Factories, seeders et Tinker

- Une **factory** fabrique des modèles avec des données aléatoires (pour les tests).
- Un **seeder** remplit la base avec un jeu de données de démonstration.

```
php artisan migrate:fresh --seed
php artisan tinker
```

```
Incident::with('site')->first()->site->nom;
Site::withCount('equipements')->get();
Liaison::first()->siteB->ville;
```

Exercices MySQL (après le seed)

Écris et exécute ces requêtes **en SQL** dans le client `mysql`, puis retrouve l'équivalent Eloquent.

1. Lister tous les sites triés par ville.
2. Lister les équipements Cisco, avec leur hostname et leur IP.
3. Compter le nombre d'équipements par type.
4. Afficher chaque équipement avec le **nom de son site** (indice : `JOIN`).

5. Afficher chaque liaison avec le code du site A et le code du site B (indice : deux `JOIN` sur la même table, avec des alias).
6. Essayer de supprimer un site qui a des équipements : quel message MySQL renvoie-t-il ? Pourquoi ?

Travail demandé

1. Ajouter les colonnes `role` et `telephone` à la migration `users`.
2. Créer les migrations `sites`, `equipements`, `liaisons`, `incidents`, `suivis`.
   - `incidents.reference` est unique (format `INC-2026-00042`) ;
   - choisir et **justifier** le comportement de suppression de chaque clé étrangère.
3. Créer les enums dans `app/Enums` : `Severite`, `StatutIncident`, `Categorie`, `Role`, `TypeSuivi`, avec `label()` et, si utile, `color()` (couleur Bootstrap).
4. Créer les modèles et **toutes** les relations.
5. Écrire le `DatabaseSeeder` : 1 admin, 4 techniciens, 8 sites, 2-3 équipements par site, 7 liaisons, ~40 incidents répartis sur les 6 dernières semaines.
6. Faire les 6 exercices MySQL et garder les requêtes dans `docs/requetes-semaine-2.sql`.

Livrable

`php artisan migrate:fresh --seed` construit une base complète et cohérente, et le fichier de requêtes SQL est commité.

Auto-évaluation

- [ ] Chaque clé étrangère a un comportement de suppression que je sais justifier
- [ ] `statut`, `severite` et `debut_at` sont indexés
- [ ] `$incident->severite` renvoie un objet `Severite`, pas une chaîne
- [ ] Toutes les relations fonctionnent dans Tinker
- [ ] Mes données de démo sont réalistes (`bko-core-rtr01`, vraies villes…)
- [ ] Je sais écrire un `JOIN` en SQL sans regarder mes notes

Ressources

- Laravel — Migrations : laravel.com/docs/migrations
- Laravel — Eloquent, relations : laravel.com/docs/eloquent-relationships
- Laravel — Factories et seeders : laravel.com/docs/seeding
- PHP — Enums : php.net/manual/fr/language.enumerations.php
- MySQL — JOIN : dev.mysql.com/doc/refman/8.4/en/join.html

Pour aller plus loin

Générer la référence de l'incident automatiquement dans l'événement `created` du modèle (indice : `static::created(...)` dans `booted()`, puis `saveQuietly()`). Pourquoi ne peut-on pas la calculer **avant** l'insertion ?

FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 3

Blade, Bootstrap et authentification

Durée : 5 jours Routes Contrôleurs Blade Bootstrap Sessions Sécurité

Objectifs de la semaine

- comprendre le trajet **route** → **contrôleur** → **vue** ;
- construire une mise en page commune avec **Blade** et **Bootstrap** ;
- écrire une **authentification** à la main et comprendre les sessions ;
- connaître trois protections de sécurité de base : CSRF, régénération de session, limitation des tentatives.

1. Routes et contrôleurs

```
// routes/web.php
Route::get('/login', [LoginController::class, 'create'])->name('login');
Route::post('/login', [LoginController::class, 'store']);
```

- `get` affiche une page, `post` reçoit un formulaire.
- `->name('login')` donne un nom à la route : dans le code on écrit `route('login')` plutôt que l'URL en dur.

```
php artisan make:controller Auth/LoginController
php artisan route:list
```

2. Blade : les gabarits

Un **layout** contient ce qui est commun à toutes les pages (menu, CSS, JS). Chaque page ne remplit que sa partie.

```
{{-- resources/views/layouts/app.blade.php --}}
<!doctype html>
<html lang="fr">
<head>
    <title>@yield('title', 'Tableau de bord')</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
    <nav class="navbar navbar-dark bg-dark">...</nav>
    <main class="container">@yield('content')</main>
</body>
</html>
```

```
{{-- resources/views/dashboard.blade.php --}}
@extends('layouts.app')

@section('content')
    <h1>Bonjour {{ auth()->user()->name }}</h1>
@endsection
```

> **Sécurité** — `{{ $variable }}` **échappe** le HTML : si quelqu'un saisit `<script>` dans un titre d'incident, il s'affichera comme du texte, sans s'exécuter. N'utilise `{!! !!}` que si tu sais exactement pourquoi.

Directives à connaître : `@if`, `@foreach`, `@forelse … @empty`, `@csrf`, `@method('PUT')`, `@error`, `@can`.

3. Bootstrap en 10 classes

Besoin	Classes
Grille	`container`, `row`, `col-md-6`, `g-3`
Carte	`card`, `card-header`, `card-body`
Tableau	`table`, `table-hover`, `table-responsive`
Boutons	`btn btn-primary`, `btn-outline-danger`, `btn-sm`
Formulaire	`form-label`, `form-control`, `form-select`, `is-invalid`, `invalid-feedback`
Badges, alertes	`badge text-bg-danger`, `alert alert-success`
Espacements	`mb-3` (marge bas), `p-4` (padding), `d-flex`, `gap-2`

4. Les composants Blade

Un composant est un petit morceau de vue réutilisable :

```
{{-- resources/views/components/badge.blade.php --}}
@props(['valeur'])
<span class="badge text-bg-{{ $valeur->color() }}">{{ $valeur->label() }}</span>
```

Utilisation : `<x-badge :valeur="$incident->severite" />`. Le `:` signifie « c'est une expression PHP ».

5. L'authentification

```
public function store(Request $request): RedirectResponse
{
    $identifiants = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (! Auth::attempt($identifiants, $request->boolean('remember'))) {
        return back()->withErrors(['email' => 'Identifiants incorrects.'])->onlyInput('email');
    }

    $request->session()->regenerate();

    return redirect()->intended(route('dashboard'));
}
```

- `Auth::attempt` cherche l'utilisateur et compare le mot de passe avec le **hash** stocké (jamais le mot de passe en clair).
- `session()->regenerate()` change l'identifiant de session à la connexion : protection contre la **fixation de session**.
- `redirect()->intended()` renvoie l'utilisateur vers la page qu'il voulait voir avant d'être redirigé vers la connexion.

Protéger des routes :

```
Route::middleware('auth')->group(function () {
    Route::get('/', DashboardController::class)->name('dashboard');
});
```

Les trois protections à connaître

1. **CSRF** : `@csrf` dans chaque formulaire `POST`. Sans lui, Laravel répond « 419 Page Expired ».
2. **Session** : régénérée à la connexion, invalidée à la déconnexion.
3. **Brute-force** : `->middleware('throttle:5,1')` sur la route de connexion = 5 essais par minute.

Exercice MySQL

Après t'être connecté, regarde la table `sessions` :

```
SELECT id, user_id, ip_address, FROM_UNIXTIME(last_activity) FROM sessions;
```

Déconnecte-toi et relance la requête. Que s'est-il passé ? Regarde aussi la colonne `password` de `users` : pourquoi ne peut-on pas retrouver le mot de passe ?

Travail demandé

1. Créer `layouts/app.blade.php` : barre de navigation (Tableau de bord, Incidents, Inventaire ▾, Rapport hebdo, Utilisateurs), nom de l'utilisateur, bouton Déconnexion, zone d'alertes flash.
2. Créer `LoginController` avec `create`, `store` et `destroy`. **Sans Breeze ni starter kit.**
3. Créer `auth/login.blade.php` (carte centrée sur fond sombre).
4. Organiser `routes/web.php` : groupe `guest` pour la connexion, groupe `auth` pour tout le reste.
5. Créer un tableau de bord provisoire qui affiche « Bonjour {nom} ».
6. Créer le composant `<x-badge>`.
7. Afficher les messages flash : `->with('success', '...')` côté contrôleur, `session('success')` dans le layout.

Livrable

Rien n'est accessible sans être connecté ; après connexion on arrive sur le tableau de bord avec le menu.

Auto-évaluation

- [ ] `/` sans être connecté redirige vers `/login`
- [ ] Mauvais mot de passe → message en français, l'e-mail reste pré-rempli
- [ ] La déconnexion se fait en `POST` avec `@csrf` (pas un simple lien)
- [ ] Après 5 essais ratés, la connexion est bloquée une minute
- [ ] Le menu indique la page active et se replie sur mobile
- [ ] Je sais expliquer à quoi sert le jeton CSRF

Ressources

- Laravel — Routing : laravel.com/docs/routing
- Laravel — Blade : laravel.com/docs/blade
- Laravel — Authentification manuelle : laravel.com/docs/authentication#authenticating-users
- Bootstrap 5.3 : getbootstrap.com/docs/5.3
- OWASP — CSRF : owasp.org/www-community/attacks/csrf

Pour aller plus loin

Ajouter « Se souvenir de moi » et expliquer le rôle du cookie `remember_token`.

FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 4

Inventaire réseau (CRUD)

Durée : 5 jours CRUD Validation Formulaires Requêtes efficaces (N+1)

Objectifs de la semaine

- maîtriser le **CRUD** Laravel (Create, Read, Update, Delete) ;
- **valider** les données reçues et afficher les erreurs proprement ;
- réutiliser un seul formulaire pour créer et modifier ;
- repérer et corriger le problème des **requêtes N+1**.

1. Les contrôleurs « resource »

```
php artisan make:controller SiteController --resource
```

```
Route::resource('sites', SiteController::class)->except('show');
```

Une seule ligne crée toutes les routes :

Verbe	URL	Méthode	Rôle
GET	/sites	index	liste
GET	/sites/create	create	formulaire vide
POST	/sites	store	enregistrer
GET	/sites/{site}/edit	edit	formulaire rempli
PUT	/sites/{site}	update	mettre à jour
DELETE	/sites/{site}	destroy	supprimer

Route model binding : en écrivant `public function edit(Site $site)`, Laravel cherche tout seul le site dont l'id est dans l'URL, et renvoie une erreur 404 s'il n'existe pas.

2. Valider les données

Règle n°1 du développement web : ne jamais faire confiance à ce qui vient du navigateur.

```
$donnees = $request->validate([
    'code' => ['required', 'string', 'max:20', Rule::unique('sites')->ignore($site)],
    'nom' => ['required', 'string', 'max:255'],
    'type' => ['required', Rule::in(Site::TYPES)],
]);

Site::create($donnees);
```

- Si la validation échoue, Laravel **revient automatiquement** au formulaire avec les erreurs et les anciennes valeurs.
- `Rule::unique('sites')->ignore($site)` : le code doit être unique, **sauf** pour le site en cours de modification.

Règles utiles cette semaine : `required`, `nullable`, `max`, `exists:sites,id`, `ip`, `different:site_a_id`, `Rule::in([...])`.

3. Un formulaire pour créer ET modifier

```
<form method="POST" action="{{ $site->exists ? route('sites.update', $site) :
route('sites.store') }}">
    @csrf
    @if ($site->exists) @method('PUT') @endif

    <input type="text" name="code" value="{{ old('code', $site->code) }}"
           class="form-control @error('code') is-invalid @enderror">
    @error('code')<div class="invalid-feedback">{{ $message }}</div>@enderror
</form>
```

- `$site->exists` est `false` pour un `new Site`, `true` pour un site venant de la base.
- `old('code', $site->code)` : après une erreur, on réaffiche ce que l'utilisateur avait tapé.
- Les formulaires HTML ne connaissent que GET et POST : `@method('PUT')` simule les autres verbes.

4. Le piège du N+1

```
@foreach ($equipements as $equipement)
    {{ $equipement->site->code }}
@endforeach
```

Avec 50 équipements, ce code exécute **51 requêtes** : 1 pour la liste, puis 1 par équipement pour aller chercher son site. C'est le problème **N+1**.

Solution, le **chargement anticipé** :

```
$equipements = Equipement::with('site')->get();   // 2 requêtes au total
```

Pour compter sans charger : `Site::withCount('equipements')->get()` ajoute un attribut `equipements_count`.

> **Astuce** — Pour voir les requêtes exécutées, ajoute temporairement dans `AppServiceProvider::boot()` : `DB::listen(fn ($q) => logger($q->sql));` puis regarde `storage/logs/laravel.log`.

Exercices MySQL

1. Écrire la requête SQL qu'exécute `Site::withCount('equipements')` (indice : sous-requête ou `LEFT JOIN … GROUP BY`).
2. Quelle différence entre `JOIN` et `LEFT JOIN` ? Ajoute un site sans équipement et compare les résultats.
3. Lister les liaisons qui partent **ou** arrivent à `BKO-DC1`.
4. Mettre à jour en SQL le statut d'un équipement à `maintenance`, puis vérifier dans l'application.

Travail demandé

1. **Sites** : liste (code, nom, ville, type, nombre d'équipements, nombre d'incidents actifs), création, modification, suppression.
   - Refuser la suppression d'un site qui a des équipements ou des incidents, avec un message clair.
2. **Équipements** : liste paginée, recherche par hostname, filtre par site ; type et constructeur en listes déroulantes ; IP validée.
3. **Liaisons** : liste « BKO-DC1 ⇄ KTI-POP1 » ; site A et site B différents ; statut (active, dégradée, coupée) en badge coloré.
4. Créer un composant `<x-bouton-supprimer>` avec confirmation JavaScript.
5. Traduire les messages de validation dans `lang/fr/validation.php`.
6. Faire un commit par écran terminé.

Livrable

Les trois écrans d'inventaire fonctionnent de bout en bout.

Auto-évaluation

- [ ] Un code de site en double est refusé à la création, accepté en modifiant le même site
- [ ] Une IP invalide (`10.10.1.300`) est refusée
- [ ] Une liaison d'un site vers lui-même est refusée
- [ ] Mes listes n'exécutent pas une requête par ligne
- [ ] La suppression demande confirmation et utilise `DELETE`
- [ ] Les messages d'erreur sont en français
- [ ] Je sais expliquer le problème N+1 avec un exemple

Ressources

- Laravel — Contrôleurs resource : laravel.com/docs/controllers#resource-controllers
- Laravel — Validation : laravel.com/docs/validation
- Laravel — Eager loading : laravel.com/docs/eloquent-relationships#eager-loading

- MySQL — LEFT JOIN : dev.mysql.com/doc/refman/8.4/en/join.html

Pour aller plus loin

Créer une page de détail d'un site avec ses équipements, ses liaisons et ses 10 derniers incidents.

FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 5

Le cœur de l'application — les incidents

Durée : 5 jours Form Request Règles métier Dates Événements Eloquent

Objectifs de la semaine

- isoler la validation dans une **Form Request** ;
- traduire des **règles métier** en code (et savoir où les placer) ;
- manipuler des dates avec **Carbon** ;
- comprendre la faille de **mass assignment** et s'en protéger.

1. Les Form Requests

Quand la validation grossit, on la sort du contrôleur :

```
php artisan make:request IncidentRequest
```

```
class IncidentRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'titre' => ['required', 'string', 'max:255'],
            'severite' => ['required', Rule::enum(Severite::class)],
            'site_id' => ['required', 'exists:sites,id'],
            'debut_at' => ['required', 'date'],
            'resolu_at' => ['nullable', 'date', 'after_or_equal:debut_at'],
            'cause_racine' => ['nullable', 'string', 'required_if:statut,resolu,clos'],
        ];
    }
}
```

Le contrôleur devient très court :

```
public function store(IncidentRequest $request): RedirectResponse
{
    $incident = Incident::create($request->validated() + ['declare_par_id' => $request->user()->id]);
    // ...
}
```

- `Rule::enum(Severite::class)` n'accepte que les valeurs de l'enum.

- `required_if:statut,resolu,clos` : obligatoire **seulement si** le statut est résolu ou clos.
- `$request->validated()` ne renvoie **que** les champs validés.

2. Mass assignment : pourquoi `declare_par_id` n'est pas dans le formulaire

Si on faisait `Incident::create($request->all())`, n'importe qui pourrait ajouter un champ caché `declare_par_id=1` avec les outils du navigateur et se faire passer pour quelqu'un d'autre.

Deux protections :

1. n'utiliser que `$request->validated()` ;
2. fixer côté serveur ce qui ne dépend pas de l'utilisateur : `'declare_par_id' => $request->user()->id`.

3. Où placer la logique métier ?

Règle	Où ?	Pourquoi
Cause racine obligatoire si résolu	Form Request	C'est une règle de **validation**
Date de rétablissement = maintenant si vide	Contrôleur (méthode privée)	Dépend de l'action de l'utilisateur
Calcul de la durée	Modèle `Incident`	Utilisé partout (liste, détail, rapport)
Référence `INC-2026-00042`	Événement du modèle	Doit exister quelle que soit la façon de créer l'incident

```
// Modèle Incident
public function dureeMinutes(): int
{
    return (int) $this->debut_at->diffInMinutes($this->resolu_at ?? now());
}
```

4. Les dates avec Carbon

Grâce au cast `'debut_at' => 'datetime'`, `$incident->debut_at` est un objet **Carbon** :

```
$incident->debut_at->format('d/m/Y H:i');   // 26/10/2026 08:30
$incident->debut_at->diffForHumans();        // il y a 3 heures
now()->subDays(30);                          // il y a 30 jours
```

Le champ HTML `datetime-local` attend le format `Y-m-d\TH:i` :

```
<input type="datetime-local" name="debut_at"
       value="{{ old('debut_at', $incident->debut_at?->format('Y-m-d\TH:i')) }}">
```

L'opérateur `?->` évite une erreur quand la date est `null` (nouvel incident).

5. L'impression

```
@media print {
    .no-print, nav.navbar { display: none !important; }
}
```

Un simple bouton `onclick="window.print()"` permet d'imprimer ou d'enregistrer en PDF la fiche d'un incident.

Exercices MySQL

1. Lister les incidents ouverts ou en cours, du plus ancien au plus récent.
2. Calculer en SQL la durée en minutes de chaque incident résolu : `TIMESTAMPDIFF(MINUTE, debut_at, resolu_at)`.
3. Trouver les incidents résolus **sans** cause racine (il ne devrait y en avoir aucun : pourquoi ?).
4. Afficher les incidents avec le nom du déclarant **et** le nom du technicien assigné (deux `JOIN` sur `users`). Que se passe-t-il pour un incident non assigné avec un `JOIN` simple ?

Règles métier à implémenter

1. Une référence unique `INC-AAAA-NNNNN` est attribuée automatiquement.
2. Le déclarant est l'utilisateur connecté.
3. Passer en **Résolu** ou **Clos** oblige à renseigner la cause racine et la solution.
4. Si on résout sans date de rétablissement, elle vaut « maintenant ». Si on réouvre, elle est effacée.
5. La date de rétablissement ne peut pas précéder le début.
6. La durée affichée est « début → rétablissement », ou « début → maintenant » si l'incident est ouvert (ex. `1j 4h 12min`).

Travail demandé

1. Créer `IncidentRequest` et `IncidentController` (resource complet).
2. Formulaire en deux colonnes : à gauche description, localisation (site, équipement, liaison), résolution ; à droite sévérité, catégorie, statut, assigné, dates.
3. Page de détail : badges, description, impact, cause et solution, colonne d'informations (site, équipement avec IP, liaison avec ses extrémités, déclarant, assigné, dates, durée).
4. Ajouter `dureeMinutes()` et `dureeLisible()` au modèle.
5. Bouton « Imprimer » avec une feuille de style d'impression.

Livrable

Un incident peut être créé, consulté, modifié et résolu, en respectant les 6 règles.

Auto-évaluation

- [ ] La référence est générée automatiquement et unique
- [ ] Résoudre sans cause racine affiche une erreur explicite en français
- [ ] Réouvrir un incident efface la date de rétablissement
- [ ] J'ai essayé d'injecter `declare_par_id` avec les outils du navigateur : sans effet
- [ ] La fiche incident s'imprime proprement
- [ ] Je sais dire pourquoi chaque règle est à l'endroit où je l'ai mise

Ressources

- Laravel — Form Requests : laravel.com/docs/validation#form-request-validation
- Laravel — Mass assignment : laravel.com/docs/eloquent#mass-assignment
- Carbon : carbon.nesbot.com/docs
- MySQL — TIMESTAMPDIFF : dev.mysql.com/doc/refman/8.4/en/date-and-time-functions.html

Pour aller plus loin

Filtrer la liste des équipements selon le site choisi, avec quelques lignes de JavaScript et un attribut `data-site` sur chaque `<option>`.

FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 6

Journal de suivi, recherche et export

Durée : 5 jours Relations Query scopes Pagination Index MySQL Export CSV

Objectifs de la semaine

- tracer l'historique d'un incident (relation `hasMany`) ;
- écrire des **requêtes réutilisables** avec les *query scopes* ;
- paginer une liste en gardant les filtres ;
- comprendre comment MySQL utilise un **index** (`EXPLAIN`) ;
- produire un fichier **CSV** téléchargeable.

1. Le journal de suivi

```
// Modèle Incident
public function suivis(): HasMany
{
    return $this->hasMany(Suivi::class)->latest();   // du plus récent au plus ancien
}
```

Créer une entrée à partir de l'incident remplit `incident_id` tout seul :

```
$incident->suivis()->create([
    'user_id' => $request->user()->id,
    'type' => TypeSuivi::ChangementStatut,
    'message' => "Statut : {$ancien->label()} → {$incident->statut->label()}",
]);
```

Pour détecter un changement de statut, on mémorise l'ancienne valeur **avant** la mise à jour.

2. Les query scopes

Un scope est un filtre réutilisable, déclaré dans le modèle :

```
public function scopeActifs(Builder $query): void
{
    $query->whereIn('statut', [StatutIncident::Ouvert, StatutIncident::EnCours]);
}
```

Utilisation : `Incident::actifs()->count()`.

Pour des filtres optionnels, `when()` n'ajoute la condition que si la valeur est renseignée :

```
public function scopeFiltrer(Builder $query, array $filtres): void
{
    $query
        ->when($filtres['statut'] ?? null, fn ($q, $statut) => $q->where('statut', $statut))
        ->when($filtres['site_id'] ?? null, fn ($q, $site) => $q->where('site_id', $site))
        ->when($filtres['q'] ?? null, fn ($q, $texte) => $q->where(
            fn ($q) => $q->where('reference', 'like', "%{$texte}%")->orWhere('titre', 'like', "%{$texte}%")
        ));
}
```

Attention aux parenthèses — Sans la fonction qui entoure le `orWhere`, MySQL lirait `statut = 'ouvert' AND reference LIKE … OR titre LIKE …` et renverrait des incidents d'autres statuts. Vérifie avec `->toRawSql()`.

Le même scope sert à la liste ET à l'export : on écrit le filtrage une seule fois (principe DRY, *Don't Repeat Yourself*).

3. La pagination

```
$incidents = Incident::with(['site', 'assigneA'])
    ->filtrer($filtres)
    ->latest('debut_at')
    ->paginate(15)
    ->withQueryString();     // garde ?statut=ouvert dans les liens de page
```

Dans la vue : `{{ $incidents->links() }}`, et dans `AppServiceProvider` : `Paginator::useBootstrapFive();`.

4. MySQL : comment un index accélère une requête

```
EXPLAIN SELECT * FROM incidents WHERE statut = 'ouvert';
```

Regarde les colonnes `type`, `key` et `rows` :

- `type = ALL`, `key = NULL` → MySQL lit **toute** la table ;
- `type = ref`, `key = incidents_statut_index` → MySQL utilise l'index et ne lit que les lignes utiles.

À l'inverse, `LIKE '%texte%'` (avec un `%` au début) ne peut **pas** utiliser un index classique : c'est acceptable pour quelques milliers de lignes, pas pour des millions.

5. L'export CSV

```
return response()->streamDownload(function () use ($incidents) {
    $sortie = fopen('php://output', 'w');
    fwrite($sortie, "\xEF\xBB\xBF");                        // BOM : Excel comprend l'UTF-8
    fputcsv($sortie, ['Référence', 'Titre', 'Statut'], ';'); // ; = séparateur Excel français
    foreach ($incidents as $incident) {
        fputcsv($sortie, [$incident->reference, $incident->titre, $incident->statut->label()],
';');
    }
    fclose($sortie);
}, 'incidents.csv');
```

`streamDownload` envoie le fichier au fur et à mesure, sans le stocker sur le disque.

Exercices MySQL

1. Lancer `EXPLAIN` sur une requête filtrée par `statut`, puis sur une requête filtrée par `categorie`. Quelle différence ? Pourquoi ?
2. Rechercher les incidents dont le titre contient « fibre ».
3. Pour chaque incident, afficher le nombre de suivis (`LEFT JOIN suivis … GROUP BY`).
4. Afficher les 5 derniers suivis toutes incidences confondues, avec la référence de l'incident et le nom de l'auteur.
5. Bonus : `SELECT … LIMIT 15 OFFSET 30` correspond à quelle page ?

Travail demandé

1. **Journal de suivi** sur la page de détail :
   - formulaire type (Commentaire / Intervention terrain) + message ;
   - affichage en timeline : icône, type, auteur, date, message ;
   - entrées automatiques à la création (« Incident déclaré ») et à chaque changement de statut.
2. **Liste des incidents** :
   - colonnes : référence, titre + catégorie, sévérité, statut, site, assigné, début, durée ;
   - filtres : recherche, statut, sévérité, site, période (du / au), bouton réinitialiser ;
   - pagination par 15 qui conserve les filtres.
3. **Export CSV** des incidents filtrés, lisible dans Excel avec les accents.

Livrable

Le journal de suivi fonctionne ; la liste se filtre, se pagine et s'exporte.

Auto-évaluation

- [ ] Un changement de statut crée bien une entrée automatique dans le journal

- [ ] Le type « Changement de statut » ne peut pas être envoyé à la main par le formulaire
- [ ] Les filtres se combinent et restent actifs en changeant de page
- [ ] La liste et l'export utilisent le même scope
- [ ] Le CSV s'ouvre dans Excel sans caractères cassés
- [ ] Je sais lire le résultat d'un `EXPLAIN`

Ressources

- Laravel — Query scopes : laravel.com/docs/eloquent#local-scopes
- Laravel — Pagination : laravel.com/docs/pagination
- Laravel — Réponses de téléchargement : laravel.com/docs/responses#file-downloads
- MySQL — EXPLAIN : dev.mysql.com/doc/refman/8.4/en/explain-output.html

Pour aller plus loin

Envoyer un e-mail (Mailable, driver `log`) au technicien quand un incident lui est assigné.

FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 7

Tableau de bord, rapport hebdomadaire et rôles

Durée : 5 jours Agrégations SQL · Collections · Dates ISO · Autorisations

Objectifs de la semaine

- calculer des **indicateurs** (comptages, moyennes) en SQL et avec les collections Laravel ;
- calculer le **MTTR** ;
- manipuler les **semaines ISO** ;
- restreindre des actions selon le **rôle** de l'utilisateur, côté vue **et** côté serveur.

1. Agréger : SQL ou collection ?

En SQL (MySQL fait le calcul, on ne récupère que le résultat) :

```
Incident::query()
    ->selectRaw('statut, count(*) as total')
    ->groupBy('statut')
    ->pluck('total', 'statut');
// ['ouvert' => 6, 'en_cours' => 2, 'resolu' => 16, 'clos' => 21]
```

Avec une collection (les lignes sont chargées en PHP, puis on calcule) :

```
$resolus = Incident::whereNotNull('resolu_at')
    ->where('resolu_at', '>=', now()->subDays(30))
    ->get(['debut_at', 'resolu_at']);

$mttr = $resolus->avg(fn (Incident $i) => $i->dureeMinutes());
```

> **Règle pratique** — Sur des milliers de lignes, faire le calcul en SQL. Sur une centaine de lignes déjà chargées pour l'affichage, une collection est plus lisible.

Compter avec une condition :

```
Site::withCount(['incidents' => fn ($q) => $q->where('debut_at', '>=', now()->subDays(30))])
    ->orderByDesc('incidents_count')
    ->limit(5)
    ->get();
```

2. Le MTTR

MTTR = somme des durées des incidents résolus ÷ nombre d'incidents résolus.

Exemple : 3 incidents de 30 min, 2 h et 4 h → (30 + 120 + 240) ÷ 3 = **130 min = 2h 10min**.

On n'inclut **que les incidents résolus** : un incident encore ouvert n'a pas de durée finale.

3. Les semaines ISO

La norme ISO 8601 numérote les semaines du **lundi au dimanche** ; la semaine 1 est celle qui contient le premier jeudi de l'année. Le champ HTML `<input type="week">` renvoie une valeur du type `2026-W44`.

```
[$annee, $numero] = explode('-W', '2026-W44');
$debut = Carbon::now()->setISODate((int) $annee, (int) $numero)->startOfWeek(); // lundi 00:00
$fin = $debut->copy()->endOfWeek();                                             // dimanche
23:59:59
```

Piège — `$debut->endOfWeek()` sans `copy()` **modifie** `$debut` ! Les objets Carbon sont modifiables ; utilise `copy()` ou `CarbonImmutable`.

Un incident « encore ouvert en fin de semaine » est un incident commencé avant la fin de la semaine **et** pas résolu à cette date :

```
->where('debut_at', '<=', $fin)
->where(fn ($q) => $q->whereNull('resolu_at')->orWhere('resolu_at', '>', $fin))
```

4. Les autorisations

Authentification = qui es-tu ? **Autorisation** = as-tu le droit ?

```
// AppServiceProvider::boot()
Gate::define('admin', fn (User $user) => $user->role === Role::Admin);
```

Trois façons de l'utiliser, à combiner :

```
Route::resource('users', UserController::class)->middleware('can:admin');   // route
Gate::authorize('admin');                                                    // dans une méthode
```

```
@can('admin')
    <a href="{{ route('users.index') }}">Utilisateurs</a>
@endcan
```

Important — Cacher un bouton ne protège **rien** : on peut toujours taper l'URL ou envoyer la requête à la main. La vérification côté serveur est obligatoire ; le `@can` dans la vue n'est que du confort.

Exercices MySQL

1. Nombre d'incidents par statut (`GROUP BY`).
2. MTTR global en minutes : `AVG(TIMESTAMPDIFF(MINUTE, debut_at, resolu_at))` sur les incidents résolus.
3. MTTR **par catégorie**, trié du plus long au plus court. Quelle catégorie coûte le plus de temps au NOC ?
4. Nombre d'incidents par semaine ISO : `YEARWEEK(debut_at, 3)` .
5. Top 3 des sites avec le plus d'incidents P1 sur les 30 derniers jours (`JOIN` , `WHERE` , `GROUP BY` , `ORDER BY` , `LIMIT`).
6. Comparer ton MTTR SQL avec celui affiché par l'application. Sont-ils identiques ?

Travail demandé

1. **Tableau de bord** :
   - 4 cartes cliquables avec le nombre d'incidents par statut (lien vers la liste filtrée) ;
   - incidents actifs par sévérité ;
   - MTTR des 30 derniers jours ;
   - top 5 des sites les plus touchés (30 jours) ;
   - P1 en cours avec leur durée, et les 8 derniers incidents.
2. **Rapport hebdomadaire** (`/rapports/hebdomadaire?semaine=2026-W44`) :
   - sélecteur de semaine (par défaut la semaine en cours) et période affichée ;
   - indicateurs : incidents de la semaine, résolus, encore ouverts en fin de semaine, MTTR ;
   - répartition par sévérité, par catégorie, par site ;
   - tableau détaillé et liste des incidents encore ouverts ;
   - version imprimable.
3. **Rôles** :
   - seul un admin gère les utilisateurs (mot de passe facultatif en modification) ;
   - seul un admin supprime un incident ;
   - un admin ne peut supprimer ni son propre compte, ni un utilisateur lié à des incidents.

Livrable

Le tableau de bord est juste, la fiche hebdomadaire s'imprime, les droits sont respectés.

Auto-évaluation

- [ ] Le MTTR affiché correspond à mon calcul à la main sur 3 incidents
- [ ] Le rapport de la semaine 43 ne contient que les incidents commencés entre le lundi 00:00 et le dimanche 23:59
- [ ] Un incident de la semaine 41 toujours ouvert apparaît dans « encore ouverts » de la semaine 43
- [ ] Un technicien qui tape `/users` reçoit une erreur 403
- [ ] La suppression d'incident est protégée côté serveur, pas seulement cachée
- [ ] Je sais expliquer la différence entre authentification et autorisation

Ressources

- Laravel — Collections : laravel.com/docs/collections
- Laravel — Autorisation (Gates) : laravel.com/docs/authorization#gates
- MySQL — Fonctions d'agrégation : dev.mysql.com/doc/refman/8.4/en/aggregate-functions.html
- MySQL — YEARWEEK : dev.mysql.com/doc/refman/8.4/en/date-and-time-functions.html

Pour aller plus loin

Ajouter un graphique (Chart.js via CDN) du nombre d'incidents par jour sur la semaine.

FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 8

Tests, finitions et soutenance

Durée : 5 jours Tests automatisés Qualité du code Documentation Présentation

Objectifs de la semaine

- écrire des **tests automatisés** qui protègent l'application ;
- formater et relire son code ;
- documenter le projet pour qu'un autre développeur puisse l'installer ;
- présenter son travail et défendre ses choix techniques.

1. Pourquoi tester ?

Sans tests, chaque modification peut casser quelque chose ailleurs sans qu'on le voie. Un test automatique vérifie en quelques millisecondes ce qu'il faudrait 10 minutes à refaire à la main.

```
php artisan make:test IncidentTest
php artisan test
```

2. Anatomie d'un test

```
class IncidentTest extends TestCase
{
    use RefreshDatabase;   // base vierge avant chaque test

    public function test_un_incident_resolu_exige_cause_et_solution(): void
    {
        // 1. Préparer
        $user = User::factory()->create();
        $incident = Incident::factory()->create();

        // 2. Agir
        $reponse = $this->actingAs($user)->put(route('incidents.update', $incident), [
            'titre' => $incident->titre,
            // ...
            'statut' => 'resolu',
        ]);

        // 3. Vérifier
        $reponse->assertSessionHasErrors(['cause_racine', 'solution']);
    }
}
```

Le schéma **Préparer / Agir / Vérifier** structure tous les tests.

Assertion	Vérifie que…
`assertOk()`	la page répond 200
`assertRedirect(route('login'))`	on est redirigé
`assertForbidden()`	l'accès est refusé (403)
`assertSessionHasErrors(['champ'])`	la validation a échoué sur ce champ
`assertSee('INC-2026')` / `assertDontSee(...)`	le texte est présent / absent
`assertDatabaseHas('incidents', [...])`	la ligne existe en base

3. Les états de factory

```
public function admin(): static
{
    return $this->state(fn () => ['role' => Role::Admin]);
}
```

`User::factory()->admin()->create()` ou `Incident::factory()->resolu()->create()` rendent les tests lisibles.

4. Les tests et la base de données

`phpunit.xml` configure les tests sur **SQLite en mémoire** :

```
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
```

Les tests sont rapides et ne touchent jamais ta base MySQL de développement. Conséquence : évite le SQL spécifique à MySQL (`YEARWEEK` , `TIMESTAMPDIFF`) dans le code de l'application, ou isole-le.

5. Qualité du code

```
vendor/bin/pint       # formate tout le code selon les conventions Laravel
```

Relis ton code avec ces questions :

- une méthode fait-elle **une seule chose** ?
- les noms disent-ils **ce que** fait le code (`dureeLisible()` plutôt que `calc2()`) ?
- y a-t-il du code copié-collé à factoriser ?
- reste-t-il des `dd()` , `dump()` ou du code commenté ?

6. Le README

Un bon README répond à : *Qu'est-ce que c'est ? Que fait-il ? Comment l'installer ? Comment se connecter ? Comment lancer les tests ?* Teste-le en suivant tes propres instructions dans un dossier neuf.

Travail demandé

1. Écrire au moins **10 tests** :
   - redirection des invités vers la connexion ;
   - connexion réussie ;
   - toutes les pages principales répondent 200 pour un admin ;
   - la création génère une référence et une entrée de suivi ;
   - la résolution exige cause et solution ;
   - la résolution renseigne la date et trace le changement de statut ;
   - ajout d'un suivi ;
   - un technicien n'accède pas aux utilisateurs et ne supprime pas d'incident ;
   - les filtres de la liste ;
   - le rapport hebdomadaire ne prend que la semaine choisie.
2. Passer Pint sur tout le projet.
3. Rédiger le `README.md` (présentation, fonctionnalités, installation, comptes de démo, tests).
4. Relire l'application comme un technicien NOC et corriger 5 irritants (libellés, alignements, messages…).
5. Préparer une démonstration de 15 minutes :

- **scénario** : « 08h12, alarme LOS sur FO-BKO-SGU » → déclaration P1 → assignation → interventions dans le journal → résolution avec cause racine → apparition dans la fiche hebdomadaire ;
- **3 choix techniques** expliqués (enums, Form Request, scope…) ;
- **ce qui reste à améliorer**.

Livrable

- `php artisan test` entièrement vert ;
- README complet et testé ;
- démonstration réalisée.

Auto-évaluation

- [ ] Au moins 10 tests, tous verts
- [ ] J'ai volontairement cassé une règle métier et vu un test échouer
- [ ] Un collègue a installé le projet avec mon seul README
- [ ] Le scénario de démonstration se déroule sans bug
- [ ] Je sais justifier mes choix et citer 3 limites de mon application

Ressources

- Laravel — Tests HTTP : laravel.com/docs/http-tests
- Laravel — Tests et base de données : laravel.com/docs/database-testing
- Laravel Pint : laravel.com/docs/pint

Pistes d'évolution

- Pièces jointes sur un incident (photos terrain, logs) avec `Storage`.
- Calcul de la **disponibilité** mensuelle d'une liaison (99,9 % ?).
- Historique de toutes les modifications d'un incident.
- Notification des P1 par SMS ou e-mail.
- API JSON pour recevoir les alarmes de l'outil de supervision (Zabbix, LibreNMS…).